

DOC-LVL26

COULOMB COUNTING DOCUMENTATION



Sanchit Joshi & Rishit Shetty
(LV)



Coulomb Counting Implementation for Low Voltage Battery Monitoring

Purpose: Accurately track battery state-of-charge (SoC) by measuring charge (Ah) flowing in and out of the LV battery.

INTRODUCTION

For our Low Voltage System we are using a 12V LiFePO4 battery. Battery Voltage will be 12V and Capacity will be 9Ah. So to track the remaining percentage of SOC and display it on the Driver's Dashboard we will be using this method. The main goal is to track the current flowing through the Power Distribution Module of the Low Voltage system and use the current sense pin to take the output as voltage and map and estimate the current flowing through the PDM. The reason why we chose coulomb counting over Voltage based calculation of SOC is because it is easier to implement and is useful for tracking real time values.

PRINCIPLE OF COULOMB COUNTING

Coulomb Counting is a battery State of Charge (SOC) estimation technique that determines the amount of charge remaining in a battery by continuously measuring the current flowing into or out of the battery and integrating it over time.

The principle is based on the fact that electric charge is the integral of current with respect to time. By tracking the charge consumed during battery discharge or the charge added during charging, the remaining battery capacity can be estimated.

The State of Charge (SOC) is calculated using:

$$SOC(t) = Q_0 - \int_0^t I(\tau) d\tau$$

Where:

- Q_0 = Initial battery charge
- $I(t)$ = Battery current at time t



- t = Elapsed time
- $SOC(t)$ = Remaining charge in the battery

The percentage State of Charge is given by:

$$SOC(\%) = \frac{SOC(t)}{Q_0} \times 100$$

Practical Implementation

In a microcontroller-based Battery Management System, the current is sampled at fixed intervals using a current sensing circuit. Since continuous integration is not feasible digitally, the integral is approximated using discrete summation:

$$SOC(t) = Q_0 - \sum_{k=1}^n I_k \Delta t$$

Where:

- I_k = Measured current during the sampling interval
- Δt = Sampling period
- n = Number of samples

In this project, the current was obtained from the current sense output of the NCV84045 smart switch. The measured current was periodically sampled by the microcontroller ADC and integrated over time to determine the charge consumed by the load.

Advantages of Coulomb Counting

- Simple and computationally efficient.
- Provides real-time SOC estimation.
- Suitable for implementation on low-cost microcontrollers such as Arduino Uno and STM32 Blue Pill.
- Does not require complex battery models.

Limitations of Coulomb Counting

- Requires an accurate initial SOC value.
- Current sensor offset errors accumulate over time.



- ADC quantization and measurement noise introduce drift.
- Periodic recalibration using methods such as Open Circuit Voltage (OCV) measurement may be required to maintain accuracy.

HARDWARE CONFIGURATION

Parameter	Values
Battery Type	Lead Acid
Manufacturer	Exide
Nominal Voltage	12V
Capacity	9Ah
Total Charge Capacity	32400As

CURRENT MEASUREMENT CIRCUIT

Current was measured using NCV84045 Smart Switch.

Parameter	Values
Sense Resistor	150Ω
Current Sense Ratio	1415
ADC Input	A0 Pin
Noise Threshold	15mV

The load current was calculated using:

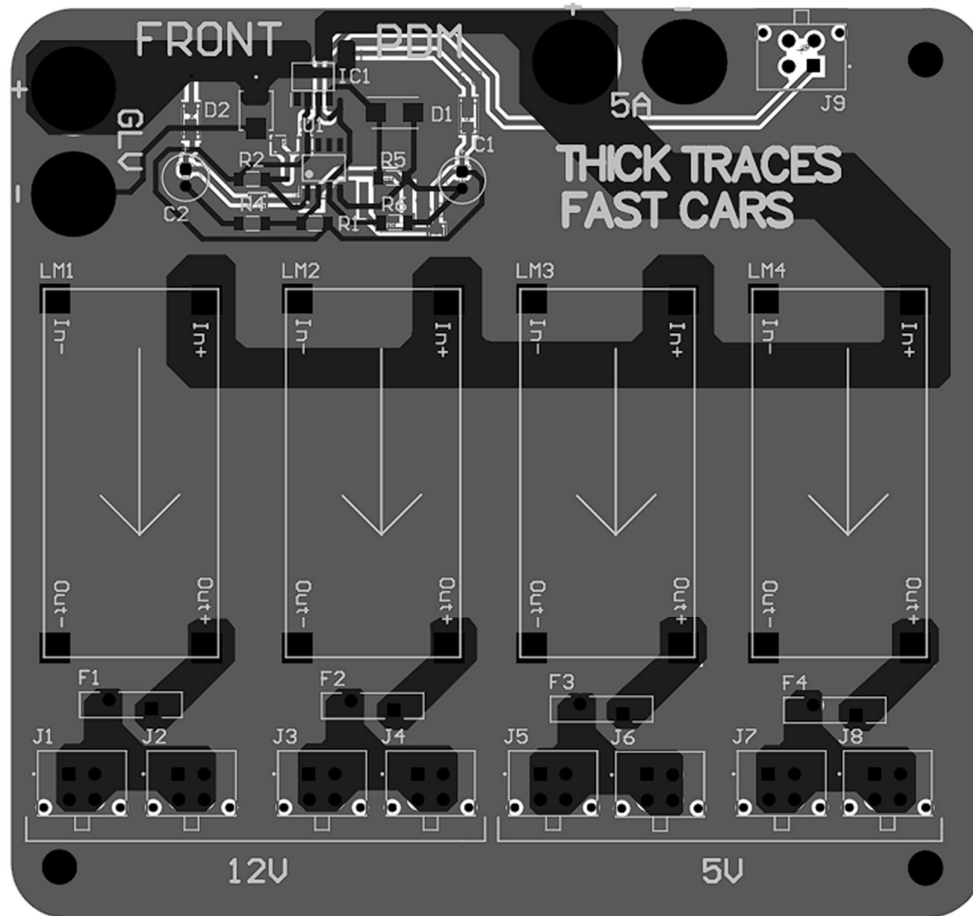
$$I = \frac{V_{CS}}{R_{SENSE}} \times K_{CS}$$

Where:

- V_{CS} = Current sense voltage
- $R_{SENSE} = 150\Omega$
- $K_{CS} = 1415$



FRONT POWER DISTRIBUTION MODULE PCB SCHEMATIC



We are using 12V power supply to simulate the battery as during the event the battery will be charged to its full potential, and due to shortage of time and with available resources, we decided to implement it using the mentioned source.

We attached one of the outputs to J9 Molex connector which is our Current Sense Pin.

We found out that if we supply the board and leave CS Pin hanging without external load, the CS pin would give us floating values, not the output that we would want.

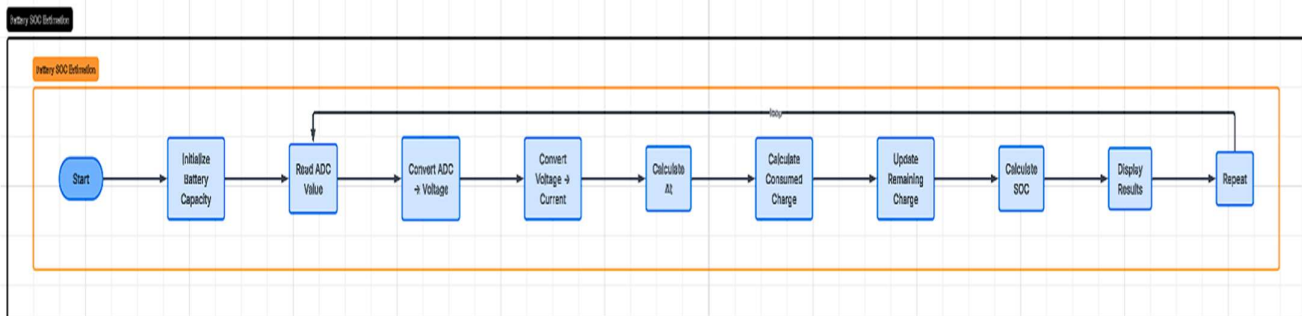
We performed multiple permutations so as to select the correct load and took outputs from the J8 Molex Connector which is a 5V output and connected it to a 5Ω 5W Power Resistor, for the reason of not having any floating values..

The CS Pin and ADC Input pin of Uno were connected in the same rail which was then connected to 5k Ω pull down resistor to GND.

We also made sure that we have common GND, which we didn't have before, giving us vague results on the serial monitor as well as live expression terminal.



SOFTWARE IMPLEMENTATION



CODE SNIPPETS

1. System Constants and Battery Parameters

Shows the battery specifications used.

```

/* Battery configuration */
#define BATTERY_CAPACITY_AH      9.0f
#define BATTERY_CAPACITY_AS      (BATTERY_CAPACITY_AH * 3600.0f)

/* ADC configuration */
#define VREF                      3.3f
#define ADC_RES                   4095.0f

/* Current-sense setup */
#define VDIV_SCALE                1.0f

/* NCV84045 current sense */
#define R_SENSE                   150.0f
#define CS_RATIO                  1415.0f

/* Noise threshold */
#define NOISE_THRESHOLD_V         0.015f
  
```

Description: Defines battery capacity, ADC characteristics, and NCV84045 current-sense parameters.



2. ADC Acquisition

```
/* — 1. Read ADC — */  
raw_adc = analogRead(ADC_PIN);  
  
/* Clamp */  
if (raw_adc > 1023)  
    raw_adc = 1023;  
  
/* — 2. Convert to voltage — */  
voltage_cs = ((float)raw_adc / ADC_RES) * VREF;
```

Description: Reads the analog current-sense signal and converts it into a voltage.

3. Noise Rejection

```
/* — 3. Noise floor — */  
if (voltage_cs < NOISE_THRESHOLD_V)  
{  
    voltage_cs = 0.0f;  
    current_a = 0.0f;  
}
```

Description: Rejects ADC noise and prevents false current measurements.

4. Current Calculation

```
/* — 4. Real current from NCV84045 — */  
current_a = (voltage_cs / R_SENSE) * CS_RATIO;
```

Description: Converts the sensed voltage into load current using the NCV84045 current-sense ratio.

Include the equation beside it:

$$I = \frac{V_{CS}}{R_{SENSE}} \times K_{CS}$$



5. Time Calculation

```
unsigned long now    = millis();
unsigned long dt_ms = now - last_tick;

if (dt_ms == 0)
    return;
```

Description: Calculates the elapsed time between consecutive measurements.

6. Coulomb Counting Core Algorithm

```
/* — 5. Coulomb counting — */
float dt_s      = (float)dt_ms / 1000.0f;
float used_charge = current_a * dt_s;

if (remaining_charge_as > used_charge)
    remaining_charge_as -= used_charge;
else
    remaining_charge_as = 0.0f;
```

Description: Computes the charge consumed during the sampling interval and updates the remaining battery charge.

Equation:

$$Q_{used} = I \times \Delta t$$



7. SOC Estimation

```
/* — 6. SoC — */  
soc_percent = (remaining_charge_as / BATTERY_CAPACITY_AS) * 100.0f;
```

Description: Calculates battery State of Charge as a percentage of the initial battery capacity.

Equation:

$$SOC = \frac{Q_{remaining}}{Q_{initial}} \times 100$$

8. Results Display

```
/* — 7. Print to Serial Monitor — */  
Serial.print(raw_adc);  
Serial.print(" | ");  
Serial.print(voltage_cs, 4);  
Serial.print("V | ");  
Serial.print(current_a, 3);  
Serial.print("A | ");  
Serial.print(soc_percent, 2);  
Serial.print("% | ");  
Serial.print(remaining_charge_as, 1);  
Serial.println(" As");
```

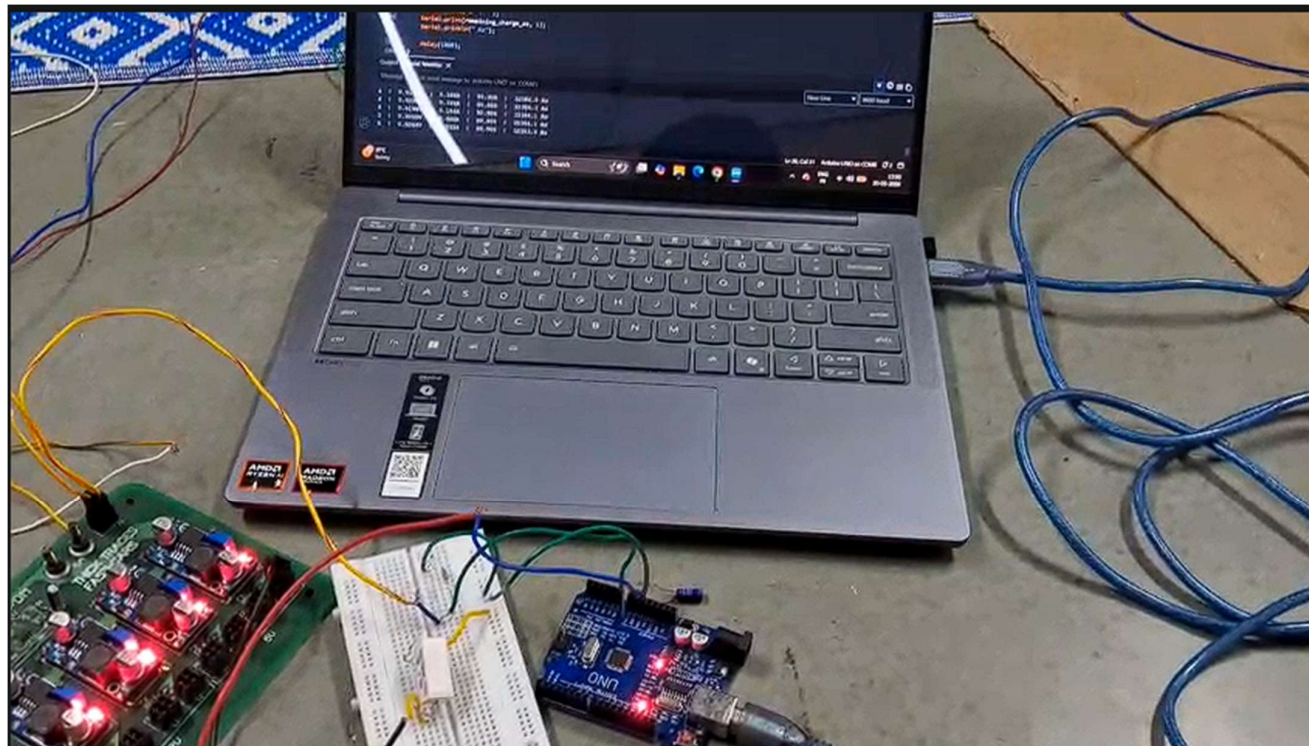
Description:

Displays real-time current and SOC values on the serial monitor.

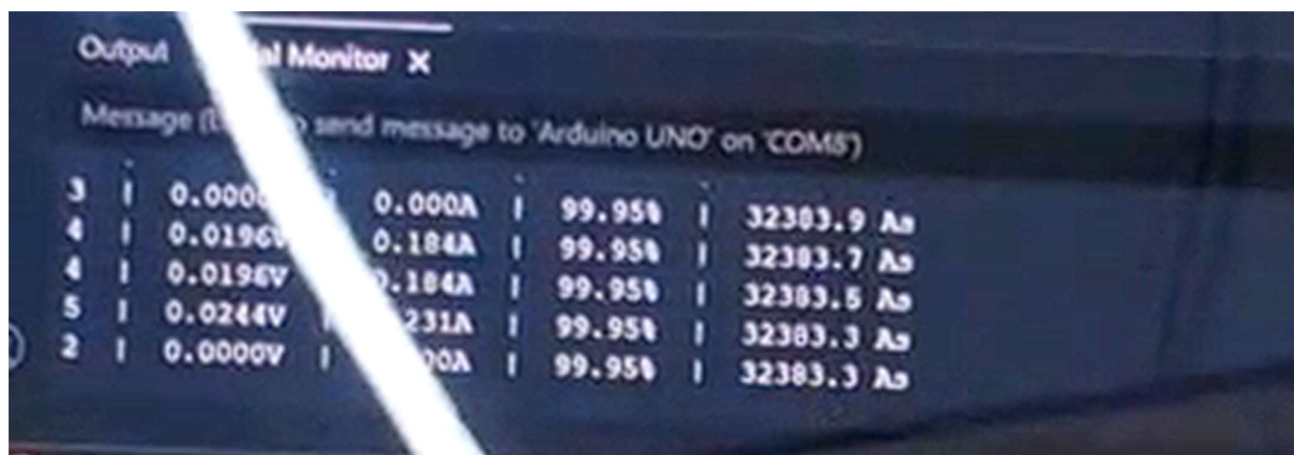


REAL TIME OBSERVATION

ON ARDUINO UNO

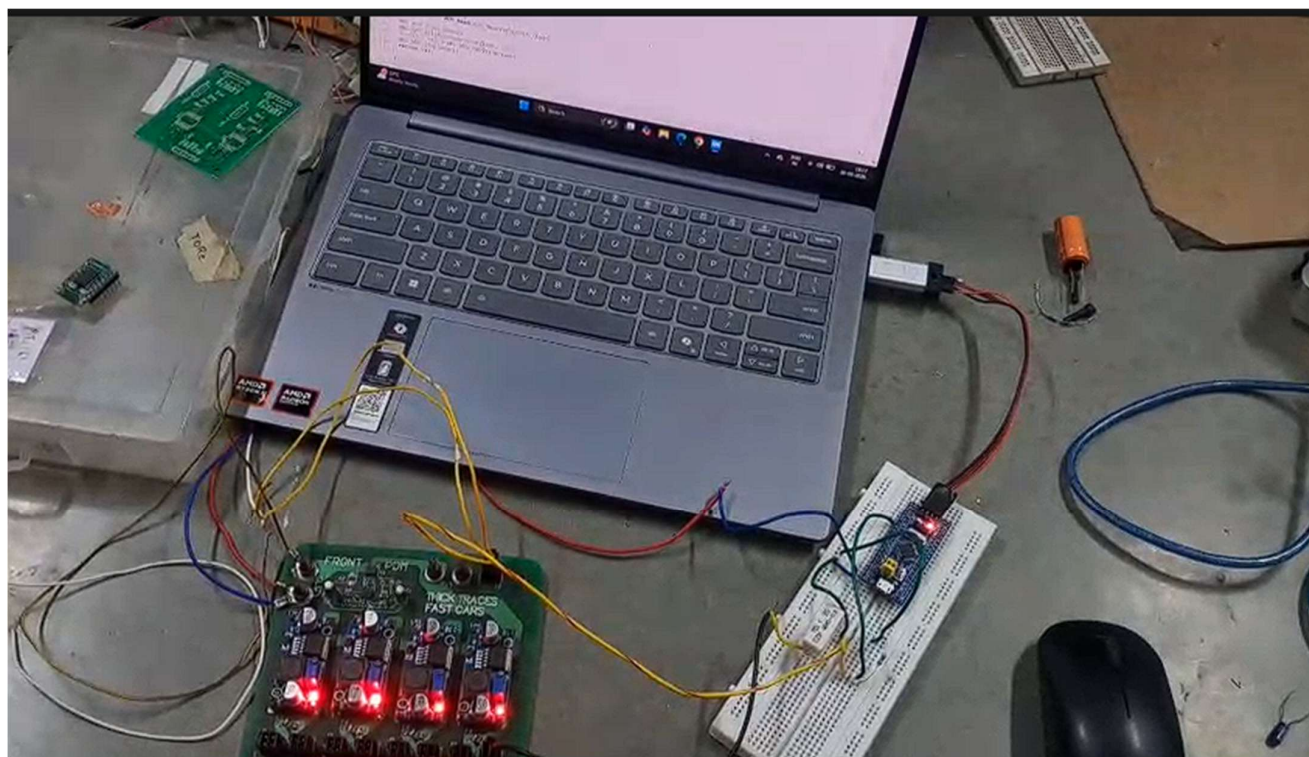


SERIAL MONITOR





ON STM32F103C8T6



LIVE EXPRESSION

Registers & Live Expressions		
Expression	Type	Value
W current_a	volatile float	1.74084735
W soc_percent	volatile float	99.4514084
W remaining_charge_as	volatile float	32727.2518
W raw_adc	volatile uint32_t	229
W actual_cs_voltage	volatile float	0.18454212
Add new expression		



Results

The implemented Coulomb Counting successfully:

- Measured load current using the NCV84045 current sense output.
- Calculated battery charge consumption in real time.
- Estimated remaining battery charge in Ampere-seconds.
- Calculated battery SOC continuously.
- Demonstrated successful operation on both Arduino Uno and STM32 Blue Pill platforms.

Future Improvements

- SOC correction using Open Circuit Voltage (OCV)
- Temperature compensation
- Coulomb counting calibration
- Kalman Filter-based SOC estimation
- Integration with complete BMS architecture
- EEPROM/Flash storage of SOC during power loss